- Make dynamic HTML and forms with ASP.NET Core easier, e.g., how many files and concepts required to print Hello World in a page, build a CRUD form, etc.
- Reduce the number of files and size of the folder structure required for page-focused MVC scenarios
- Simplify the code required to implement common page-focused patterns, e.g., dynamic pages, CRUD forms, etc.
- Enable the ability to return non-HTML responses when necessary, e.g., 404s
- Use and expose the existing MVC primitives (components) as much as possible

Ultimately, Razor Pages was introduced to make working with the MVC pattern simpler than the existing framework. That is not to say that Razor Pages is only suitable for simple scenarios—far from it, although that is a view you might find on various websites. However, when pressed, you find that most folk with that view admit to not having tried Razor Pages.

1.4 When should you use Razor Pages?

In keeping with my claim that it's probably easier to list the things you can't do with Razor Pages, I'll start this section by looking at examples of when you shouldn't consider using Razor Pages:

- *Single-page applications*—Being a server-side development framework, Razor Pages is not a suitable tool for building single-page applications, in which the application is usually written in JavaScript and executes in the browser, except where server rendering is a requirement (http://mng.bz/YGWB).